

Pre-work for *Sunday's session.*

Most builders who say they want to ship with AI never actually do. They install Claude Code, they watch a demo or two, and then they stare at the cursor when the moment comes to make something real. This Sunday is built for people who would rather be on the other side of that line by the time the session ends. In two hours of guided building, you will deploy two real products to two URLs of your own choosing, and you will walk away with the build loop sitting comfortably in your head, ready to use on Tuesday morning the next time you have something of your own to ship.

What I will tell you about Sunday

The session runs for two hours and splits cleanly into two halves of roughly equal weight. In the first half, which takes about fifty-five minutes including a short warm-up, I will build a real product live in front of you, going from an empty folder on my laptop to a deployed URL in about thirty-five minutes of focused work, while three subagents handle different parts of the codebase in parallel. I will narrate what is happening at each step rather than dictate every keystroke, because the lesson here is the build loop and not the typing itself. The product I am building is small enough to ship in that window, but it is the kind of thing I expect you to find yourself opening on Tuesday morning the next time you need to think out loud about a hard decision or want a second pair of eyes on something you wrote.

In the second half you build your own real product from your own empty folder, following the same loop I demonstrated. The wiring you write is identical to the wiring the person sitting next to you is writing, but the personality you put into your CLAUDE.md file is what makes your version completely different from theirs. If Daisy the PM is in the room, her product will sound like a senior PM giving feedback. If Marco the engineer is two seats over, his product will catch passive voice with a vengeance. If Priya the designer is at the back, her product will edit like a magazine art director. Fifteen people in the room will deploy fifteen distinct products from the same skeleton, and the showcase at the end of the session is the moment when we click through each other's URLs and notice how completely different they feel even though the code underneath is identical.

What I am keeping back as a surprise for the live session is the shape of those two products. I am doing this deliberately, because the value of the session lives in the loop you learn rather than in the specific artifacts we ship, and because the reveal lands better when you see it happen in front of you on Sunday than when you read about it in a doc on Thursday. If you want a hint about what is coming, the responses you get from your deployed URLs on Sunday afternoon will sound like you, and the file that makes them sound like you is the same CLAUDE.md you are about to write during the pre-work below.

Pre-flight checklist

Each item below has a verify step that lets you confirm the piece is working before you move on to the next one. The whole sequence ends with a single script that checks everything at once, and you should run that script after you finish each individual step. Going through this pre-work is non-negotiable for the session; this is not a session where you can show up unprepared and absorb the lesson by osmosis.

1. Claude Code (VS Code is primary)

- ☐ Install the **Claude Code** extension from the VS Code Marketplace. Sign in with your Pro or Max Anthropic account.

VERIFY Open VS Code, hit `⌘⇧P`, search "Claude Code: New session." A side panel opens.

- ☐ Confirm the CLI also works: `cClaude` in your terminal.

VERIFY The Claude Code REPL launches.

2. Tooling

- ☐ **Node.js 20 or higher.** `node --version`.

VERIFY `v20.x.x` or above.

- ☐ **pnpm.** Install with `npm i -g pnpm` if missing. Then `pnpm --version`.

VERIFY `9.x` or above.

☐ **GitHub CLI.** `gh auth status` .

VERIFY Shows you logged in to `github.com` .

☐ **Vercel CLI.** Install with `npm i -g vercel` if missing. Then `vercel whoami` .

VERIFY Prints your Vercel username.

3. MCPs and skills

Inside Claude Code, install the six MCPs and the superpowers plugin:

```
claude mcp add claude-mem
claude mcp add context7
claude mcp add stitch
claude mcp add shadcn
claude mcp add vercel
claude mcp add github

claude plugins add superpowers
```

☐ **VERIFY** `claude mcp list` shows all six. `claude plugins list` shows superpowers.

4. Customize your CLAUDE.md

This is the most important step in the entire pre-work, and the one that deserves the most thought. Your CLAUDE.md is the file that tells Claude Code how you work and what you care about, and it also quietly does a second job during Sunday's session that I am keeping back as part of the surprise. The more honest and specific you make this file, the better your Sunday will go, because vague preferences produce vague behavior, while specific preferences produce a tool that feels like a genuine extension of how you think.

The kit at `claude-md-kit/` gives you two ways to build the file. Path A is the recommended one, and it works like this: you open Claude Code in a fresh session, you paste the prompt from `claude-md-kit/INTERVIEW.md` , and you answer the questions Claude asks you section by section. The whole interview takes about ten to fifteen minutes and produces a file in your own voice rather than a translation of someone else's template. Path B is for people who would rather write markdown directly: you copy the spine template into your editor and fill in each section yourself, using the walkthrough in `claude-md-kit/CUSTOMIZATION.md` for

guidance on what each section is asking for. Either path is fine for the session, but the file you produce should sound nothing like mine, because the entire point is that yours is yours.

- ☐ Clone the enablement repo so you have the kit on disk:

```
git clone https://github.com/v60samurai/rethink-enablement
```

- ☐ Use Path A or Path B from above to create your `~/ .claude/CLAUDE.md` . The interview path is faster and produces a file in your own voice.

VERIFY *Open Claude Code, type summarize my CLAUDE.md. The summary should match what you wrote (not what someone else wrote).*

5. Validation script

One command checks all of the above:

```
bash rethink-enablement/session-plan/validate-prework.sh
```

A row of green check marks means you are ready for Sunday. Any cross means the line above it tells you what is still missing, and the message points at the install command you need to run.

One more thing before you start

The worst-case outcome of doing everything in the pre-work above is that you arrive on Sunday with a working setup and you ship at least one real product to a real URL before the session ends. The best-case outcome is that you ship both of them, that your friends ask you on Monday morning how you did it, and that the build loop you learned during the session becomes the loop you reach for the next time you have something of your own to build on the side or at work.

The worst-case outcome of skipping the pre-work, on the other hand, is that you spend Sunday installing tools and fixing config errors while the rest of the room ships products to live URLs, and you walk out at the end of the session with the same intention to build that you

walked in with. Reading about other people's URLs in the workshop chat afterwards is a worse experience than dropping your own URL in alongside theirs, and the only thing standing between you and the second experience is the twenty minutes of setup work above.

See you Sunday.